

Documentazione AbstractService<ENTITY, DTO>

Scopo della classe

AbstractService è una classe astratta e generica che implementa l'interfaccia **ServiceDto**. Il suo obiettivo è fornire una implementazione comune delle principali operazioni CRUD (Create, Read, Update, Delete) evitando di riscrivere lo stesso codice in ogni Service dell'applicazione.

La classe utilizza due tipi generici: **ENTITY** (rappresenta l'entità JPA salvata nel database) e **DTO** (rappresenta l'oggetto trasferito tra Controller e client). Per effettuare la conversione tra Entity e DTO utilizza un **Converter**, mentre per l'accesso ai dati utilizza **JpaRepository**.

Attributi

- **repository**: è il repository JPA che comunica con il database per eseguire le operazioni CRUD.
- **converter**: è il componente responsabile della conversione bidirezionale tra Entity e DTO.

Costruttore

Il costruttore riceve tramite Dependency Injection il **JpaRepository** e il **Converter**. Entrambi vengono salvati negli attributi della classe e saranno utilizzati da tutti i metodi CRUD ereditati dalle classi concrete.

Metodi

insert(DTO dto)

Questo metodo inserisce un nuovo record nel database. Riceve un DTO dal Controller, lo converte in Entity tramite **converter.toEntity()** e lo salva nel database con **repository.save()**. Hibernate restituisce l'Entity aggiornata con l'ID generato, che viene riconvertita in DTO tramite **converter.toDTO()** e restituita al Controller.

Flusso: Controller → DTO → Converter → Entity → Repository.save() → Database → Entity → Converter → DTO → Controller.

getAll()

Restituisce tutti i record presenti nel database. Il metodo chiama **repository.findAll()** per recuperare la lista completa di Entity, poi utilizza **converter.toDTOList()** per convertire la lista in oggetti DTO e la restituisce al Controller.

Flusso: Controller → Repository.findAll() → Database → List<Entity> → Converter → List<DTO> → Controller.

read(Integer id)

Ricerca un record specifico tramite il suo identificativo univoco. Riceve l'ID in ingresso ed esegue **repository.findById(id)**. Estrae l'Entity dall'**Optional** tramite **.get()**, la converte in DTO e restituisce il risultato.

Nota: l'uso diretto di **.get()** può generare **NoSuchElementException** se il record non esiste; in un ambiente di produzione è consigliabile gestire il caso con **orElseThrow()**.

update(DTO dto)

Aggiorna un record esistente. Riceve il DTO contenente i dati aggiornati, lo converte in Entity e chiama **repository.save()** per persistere le modifiche. L'Entity aggiornata viene riconvertita in DTO e restituita. In JPA il metodo **save()** gestisce sia inserimenti che aggiornamenti: se l'ID è presente aggiorna il record, altrimenti ne crea uno nuovo.

delete(Integer id)

Elimina un record dal database tramite il suo identificativo primario ricevendo l'ID ed eseguendo **repository.deleteById(id)**. Il metodo non restituisce alcun valore.

Vantaggi della AbstractService

Elimina la duplicazione del codice CRUD, centralizza la logica comune a tutti i Service, favorisce il riutilizzo del codice, riduce gli errori di implementazione e consente alle classi concrete di focalizzarsi solo sulla logica di business specifica.